1 **Design & Implementation of Human-Computer Interfaces**
NPTEL-MOOCS

Dr Samit Bhattacharya
Computer Science and Engineering
IIT Guwahati

2 **L22 – Introduction to UML**

Dr Samit Bhattacharya
Computer Science and Engineering
IIT Guwahati

3

4 **Basic Design Approaches - Recap**

- Function oriented – basic abstractions are functions
- Object oriented – basic abstractions are objects (instantiation of class)

5 **Basic Design Approaches - Recap**

- Earlier discussed basic idea of OOD

6 **Recap**

- In this lecture, we shall discuss UML (to represent object-oriented design)

7 **UML**

8 **UML (Unified Modeling Language)**

- Like DFD, we need a "language" to represent OOD
  - UML provides that
- May be used to visualize, specify, construct, and document artifacts of a software system

9 **UML (Unified Modeling Language)**

- Provides a set of notations (e.g. rectangles, lines, ellipses, etc.) to create a visual/graphical model of the system
- Like any other language, has its own syntax (symbols and sentence formation rules) and semantics (meanings of symbols and sentences)

10 **Note**

- UML not a design methodology
  - Only a language to express object-oriented design obtained using some methodology

11 **Views & Diagrams**

- NINE diagrams to capture FIVE views of the system

12 **Views & Diagrams**

—User view (use case diagrams)
—Behavioral view (sequence diagram, collaboration diagram, state chart diagram, activity diagram)
—Structural view (class diagram, object diagram)
—Implementation view (component diagram)
—Environment view (deployment diagram)

13 **Views & Diagrams**

—User view (use case diagrams)
—Behavioral view (sequence diagram, collaboration diagram, state chart diagram, activity diagram)
—Structural view (class diagram, object diagram)
—Implementation view (component diagram)
—Environment view (deployment diagram)

14

User View

15 **Use Case Diagram (User View)**

- Idea: to represent user perception of the system (with dialog/conversation between user and system to express 'interaction')

16 **Use Case Diagram (User View)**

- Basic components
  - Actor (stick figure)
  - System boundary
  - Use case (ellipse)
  - Mainline sequence & alternative sequence

17

18

19

Behavioral View

20 **After Use Case**

- What to do after use case identification?
  - Identify objects and classes and how they interact

21 **After Use Case**

- How to do that?
  - Experience, intuition, domain knowledge …
  - Some systematic approach!

22 **Domain Modeling**

- A simple and systematic approach to determine classes and objects
- Idea - identify obvious domain objects and their relations

23 **Domain Objects**

- Boundary objects – those with which the actors interact (screen, menu etc); can be identified from use cases
- Controller objects – coordinate activities between boundary and entity objects
- Entity objects – normally hold information (e.g., book, book registers etc)

24 **Note**

- Key thing to do – identify domain objects for each use case
  - There may be many similar objects – remove repetition

25 **Interaction Diagram**

- Next create an interaction diagram for each use case execution scenario
  - To refine list of objects

- Also to identify classes by clubbing together similar objects

26 **Interaction Diagram**

- Captures dynamic system behavior - typically one diagram per use case
- Many types
  - —Sequence diagram
  - —Collaboration diagram
  - —Activity diagram …

27 **Interaction Diagram**

- Captures dynamic system behavior - typically one diagram per use case
- Many types
  - —Sequence diagram
  - —Collaboration diagram
  - —Activity diagram …

28 **Interaction Diagram**

29 **Recap**

- OOD sequence: Use case analysis ☐ domain modeling ☐ sequence diagram
  - Help to identify objects, classes and their behavior (over time)
  - Needs many iterations and experience
  - Easier said than done!

30 **What's Next?**

- We need to represent "structure" of the design
  - Classes and their relationship

31

Structural View

32 **Class Diagram**

- Describes static system structure
- Comprised of classes and their relationship

33 **Class Representation**

- Name: AnyInteger
- Attribute: integerValue
- Operation/member function: convertToCharacter [input = integer, output = Unicode value]

34 **Class Relationships**

- Association
- Aggregation (whole-part relationship)
- Composition (a stricter form of aggregation)

35 **'Association' Relationship**

- Describes connection between classes
- Links – instances of association between two objects
  - Ex - Ramesh borrowed book "Operating System" - borrowed is the link between objects Ramesh (instance of class 'student') and Operating System (instance of class 'book')
- Association describes a group of links with common structure and common semantics

36 **'Association' Relationship**

- Usually, association is a binary relation (between two classes)
  - However, 3/more classes can be involved in an association
- A class can have an association relationship with itself (recursive association)
  - Assumption - two different objects of the class are linked by the association relationship

37 **Representing 'Association'**

- By a straight line between classes - name written along side line
- An arrowhead may be placed to indicate direction of association
- On each side of line, multiplicity is noted
  - Indicates no of instances of one class associated with other
  - Value ranges noted by specifying minimum and maximum, separated by two dots (e.g., 2..4)
  - An asterisk is a wild card and means many (zero or more)

38 **'Aggregation' Relationship**

- A special type of association - involved classes represent a whole-part relationship
- Represented by diamond symbol at the 'whole' end of a relationship

39 **'Composition' Relationship**

- A stricter form of aggregation - parts are existence-dependent on the whole
  - When whole is created, parts are created and when whole is destroyed, parts are destroyed
- Represented as a filled diamond drawn at the 'whole' end

40

41 **Putting Everything Together: OOD Process**

42 **Reference**

- Rajib Mall (2018). Fundamentals of Software Engineering, 5th ed, PHI Learning Pvt Ltd. Chapter 7